Multi-Agent Reinforcement Learning for Autonomous Rescue Drone Coordination

Project Argus

Deep Patel*, Teammate 2*, Teammate 3*
*Texas A&M University - College Station
Email: {dpate149, author2, author3}@tamu.edu

***Abstract*—This paper presents a multi-agent reinforcement learning approach for coordinating autonomous rescue drones in disaster scenarios. We implement and compare Proximal Policy Optimization (PPO), a Graph Neural Network (GNN) encoder and Deep Q-Network (DQN) agents in grid-based and continous rescue environments. Our results demonstrate that agents with properly designed observation spaces and reward shaping can effectively learn cooperative rescue strategies, achieving an average of X survivors rescued per episode with Y% efficiency improvement over baseline approaches.**

***Index Terms*—Multi-agent reinforcement learning, rescue robotics, PPO, GNN, DQN autonomous drones, disaster response**

I. Introduction

In disaster response scenarios, coordinating multiple autonomous rescue drones presents significant challenges in terms of efficient search patterns, resource allocation, and survivor detection [1]. Recent advances in reinforcement learning (RL) offer promising solutions for enabling autonomous agents to learn optimal coordination strategies [2].

This work presents **Project Argus**, a multi-agent RL framework for training rescue drones to:

- Navigate complex grid-based environments
- Detect and rescue survivors efficiently
- Coordinate with other agents to maximize coverage
- Adapt to varying environment configurations

Our main contributions are:

1) A comparative analysis of PPO and DQN algorithms in rescue scenarios
2) Novel observation space design incorporating relative positioning
3) Reward shaping techniques for sparse-reward environments
4) Empirical evaluation demonstrating X% improvement over baselines

II. Related Work

A. Multi-Agent Reinforcement Learning

Brief overview of MARL techniques and their applications [3].

B. Rescue Robotics

Review of existing work in autonomous rescue systems [4].

C. Reward Shaping

Discussion of reward engineering in sparse-reward environments [5].

III. Methodology

To build a rigorous and comprehensive framework, we constructed a diverse collection of environments and multi-agent models. This collection covered the full specturm from lightweight, discrete enivornments to high-complexity, continous systems.

A. Grid-based Environment Baseline

1) Environment Design: We developed a grid-based simulation environment using the PettingZoo framework [6]. The environment consists of:

- **Grid Size:** 10×10 cells
- **Agents:** 1-3 rescue drones
- **Survivors:** 2-3 randomly placed per episode
- **Actions:** Discrete movements (up, down, left, right)

2) Observation Space: Each agent receives a 4-dimensional observation vector:

$$\mathbf{o}_t = [x_{\text{norm}}, y_{\text{norm}}, \Delta x_{\text{surv}}, \Delta y_{\text{surv}}] \tag{1}$$

where $x_{\text{norm}}, y_{\text{norm}}$ are normalized agent coordinates, and $\Delta x_{\text{surv}}, \Delta y_{\text{surv}}$ represent the relative direction to the nearest survivor.

3) Reward Function: Our reward function comprises three components:

$$r_t = r_{\text{rescue}} + r_{\text{distance}} + r_{\text{time}} \tag{2}$$

- **Rescue Reward:** $r_{\text{rescue}} = +10.0$ when rescuing a survivor
- **Distance Reward:** $r_{\text{distance}} = (d_{t-1} - d_t) \times 0.1$
- **Time Penalty:** $r_{\text{time}} = -0.01$ per timestep

This design encourages efficient pathfinding while providing dense feedback for learning.

4) Algorithm: Proximal Policy Optimization: We employ PPO with an Actor-Critic architecture:

Actor Network: Policy $\pi_\theta(a|s)$

$$\text{Input}(4) \to \text{FC}(64) \to \text{FC}(64) \to \text{Softmax}(4) \tag{3}$$

Critic Network: Value function $V_\phi(s)$

$$\text{Input}(4) \to \text{FC}(64) \to \text{FC}(64) \to \text{Output}(1) \tag{4}$$

PPO Objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{5}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the advantage estimate.

B. Continous Geospatial Enviornment

1) Environment Design: We constructed a geospatial multi-agent environment grounded in real-world geographic data. The environment uses OpenStreetMap to pull features such as buildings, water bodies, parks, and terrain structures—extracted within a specified bounding region. It is then projected into a metric coordinate system (EPSG:3857) and all elements are converted into a unified planar representation, enabling continuous navigation and distance calculations.

Agents operate in a fully continuous 2D workspace defined as

$$\mathcal{X} = [0, M_x] \times [0, M_y] \subset \mathbb{R}^2,$$

where M_x, M_y denote the map's metric extents. Each agent $i \in \{1, \ldots, N\}$ has a state

$$s_i(t) = \big(x_i(t),\, y_i(t),\, b_i(t)\big),$$

where $(x_i(t), y_i(t)) \in \mathcal{X}$ is its position and $b_i(t)$ its battery level. Agents are initialized within a small neighborhood of the origin,

$$(x_i(0),\, y_i(0)) \sim \text{Uniform}([0,\, lM_x] \times [0,\, wM_y])\,.$$

where l and w allow the restriction of a 2-D surface where the agents can be distributed.

Survivor locations are sampled uniformly from $\mathcal{X}$:

$$p_k \sim \text{Uniform}(\mathcal{X}), \qquad k = 1, \ldots, S,$$

where the total number of survivors satisfies

$$S \sim \text{Uniform}(\alpha(M_x + M_y),\ \beta(M_x + M_y))\,.$$

α and β are tunable parameters to control the distribution of surviviors on the map.

All agents act simultaneously in a shared environment implemented through PettingZoo's `ParallelEnv` interface,

$$\mathcal{E}:\ \big(a_1(t), \ldots, a_N(t)\big) \ \longmapsto\ \big(O_1(t+1), \ldots, O_N(t+1)\big).$$

2) Observation Space: Each agent i receives an observation $O_i(t)$ drawn from a fixed observation space

$$\mathcal{O}_i = \mathbb{R}^{K \times 5},$$

where $K = 5$ denotes the memory limit of the environment. Formally,

$$O_i(t) = \begin{bmatrix} o_{i,1}(t) \\ o_{i,2}(t) \\ \vdots \\ o_{i,K}(t) \end{bmatrix} \in \mathbb{R}^{K \times 5}.$$

Each row $o_{i,j}(t) \in \mathbb{R}^5$ encodes the attributes of the j-th nearest entity to agent i at time t. Let the agent's position be

$$(x_i(t),\, y_i(t)) \in \mathcal{X},$$

and let the j-th entity have global coordinates

$$(x_{i,j}(t),\, y_{i,j}(t)).$$

The relative spatial offsets are defined as

$$\Delta x_{i,j}(t) = x_{i,j}(t) - x_i(t), \qquad \Delta y_{i,j}(t) = y_{i,j}(t) - y_i(t),$$

and the Euclidean distance to the entity is

$$d_{i,j}(t) = \left\| \begin{bmatrix} \Delta x_{i,j}(t) \\ \Delta y_{i,j}(t) \end{bmatrix} \right\|_2 = \sqrt{\Delta x_{i,j}(t)^2 + \Delta y_{i,j}(t)^2}.$$

The complete observation vector for the j-th entity is therefore defined as

$$o_{i,j}(t) = \big(b_i(t),\, \Delta x_{i,j}(t),\, \Delta y_{i,j}(t),\, d_{i,j}(t),\, \tau_{i,j}\big),$$

where $b_i(t) \in [0, B_{\max}]$ denotes the agent's remaining battery and $\tau_{i,j} \in \mathcal{T}$ is a categorical label indicating the semantic type of the entity (e.g., building, water, park, terrain), with

$$\mathcal{T} = \{1, 2, 3, 4\}.$$

If fewer than K entities fall within the agent's sensing field, the corresponding rows of $O_i(t)$ are populated with zero vectors, i.e.,

$$o_{i,j}(t) = \mathbf{0} \quad \text{for all } j > n_i(t),$$

where $n_i(t)$ is the number of entities detected at time t.

Thus, the full observation provided to agent i at time t is the structured matrix

$$O_i(t) = \big[\, o_{i,1}(t),\ o_{i,2}(t),\ \ldots,\ o_{i,K}(t) \,\big]^\top \in \mathbb{R}^{K \times 5},$$

representing the agent's battery status together with the relative spatial and semantic properties of its K nearest environmental entities.

3) Reward Function: Each agent i receives a scalar reward $r_i(t) \in \mathbb{R}$ at each time step t based on its actions and interactions with the environment. The reward function is defined as

$$r_i(t) = r_i^{\text{prox}}(t) + r_i^{\text{check}}(t) + r_i^{\text{time}} + r_i^{\text{oob}}(t),$$

where each term captures a distinct contribution:

1) **Proximity reward:** Let $p^*(t) \in \mathcal{P}(t)$ be the nearest survivor to agent i. Denoting the agent's current and next positions as $p_i(t)$ and $p_i(t+1)$, the proximity reward is
$$r_i^{\text{prox}}(t) = \frac{\|p^* - p_i(t)\|_2 - \|p^* - p_i(t+1)\|_2}{D_{\max}},$$
where $D_{\max} = \sqrt{M_x^2 + M_y^2}$ normalizes the reward by the maximum possible diagonal distance in the map.
2) **Check-for-survivor reward:** If the agent chooses to check ($c_i(t) = 1$) and detects $n_i(t)$ survivors within a local sensing radius r, it receives
$$r_i^{\text{check}}(t) = 10 \cdot n_i(t),$$
and the detected survivors are removed from the global survivor set $\mathcal{P}(t)$.
3) **Time penalty:** To encourage efficient exploration,
$$r_i^{\text{time}} = -0.01.$$
4) **Out-of-bounds penalty:** If the agent attempts to move outside the map bounds,
$$r_i^{\text{oob}}(t) = \begin{cases} -0.5, & \text{if } (x_i(t+1), y_i(t+1)) \notin \mathcal{X}, \\ 0, & \text{otherwise.} \end{cases}$$

The total reward thus integrates progress toward survivors, successful detection actions, efficiency, and safety.

Finally, a small terminal bonus is provided if all survivors are located:

$$r_i(t) \to r_i(t) + 0.1 \quad \text{if } |\mathcal{P}(t)| = 0.$$

This formulation encourages agents to move efficiently toward survivors, coordinate detection actions, and avoid invalid moves while navigating the geospatial environment.

4) GNN-based Multi-Agent Model: Each agent $i \in \{1, \ldots, N\}$ is parameterized by a Graph Neural Network (GNN) policy π_θ and a value head V_ϕ to enable cooperative multi-agent reasoning. Observations from the environment at time t are aggregated into a graph representation:

$$\mathcal{G}(t) = (\mathcal{V}, \mathcal{E}, \mathbf{X}(t)),$$

where $\mathcal{V} = \{1, \ldots, N\}$ denotes the set of agents, $\mathcal{E} = \{(i,j) \mid i \neq j\}$ represents a fully connected communication graph, and $\mathbf{X}(t) \in \mathbb{R}^{N \times d_{\text{obs}}}$ contains per-agent normalized observations:

$$\mathbf{X}_i(t) = \frac{\mathbf{o}_i(t) - \mu_i}{\sigma_i}, \quad i = 1, \ldots, N,$$

with μ_i and σ_i the mean and standard deviation of agent i's observation vector.

a) GNN Encoder: The graph features are passed through a two-layer GNN:

$$\mathbf{H}^{(1)} = \sigma\big(\text{GCN}(\mathbf{X}, \mathcal{E})\big), \qquad \mathbf{H}^{(2)} = \sigma\big(\text{GraphSAGE}(\mathbf{H}^{(1)}, \mathcal{E})\big),$$

where σ is the ReLU activation, and $\mathbf{H}^{(2)} \in \mathbb{R}^{N \times d_{\text{hid}}}$ is the learned agent embedding.

b) Policy and Value Heads: Each embedding $\mathbf{h}_i \in \mathbf{H}^{(2)}$ is passed through a shared multi-layer perceptron (MLP) to compute:

$$\boldsymbol{\mu}_i = f_\theta(\mathbf{h}_i) \in \mathbb{R}^{d_{\text{act}}},$$
$$V_i = f_\phi(\mathbf{h}_i) \in \mathbb{R}.$$

Here, $\boldsymbol{\mu}_i$ represents the mean of the Gaussian policy for continuous actions, and V_i is the value estimate for actor-critic training. The policy standard deviation is parameterized as a learnable vector $\log \boldsymbol{\sigma} \in \mathbb{R}^{d_{\text{act}}}$.

c) Graph Construction: Observations are converted into a fully-connected graph excluding self-loops:

$$\mathcal{E} = \{(i,j) \mid i, j \in \mathcal{V}, i \neq j\}.$$

The resulting graph $\mathcal{G}(t)$ is fed into the GNN encoder at each time step to produce coordinated action outputs for all agents simultaneously.

IV. Experiments

A. Experimental Setup

We evaluate our approach across three configurations:

1) Single-agent rescue (baseline)
2) Multi-agent with random observations (ablation)
3) Multi-agent with engineered observations (full system)

B. Evaluation Metrics

- Average survivors rescued per episode
- Episode reward (cumulative)
- Average episode length
- Success rate (all survivors found)

V. Results

A. Training Performance

Figure 1 shows the learning curves for PPO agents over 1000 episodes.

B. Training Performance

Figure 1 shows the learning curves for PPO agents over 1000 episodes.

C. Quantitative Results

Table I presents the performance comparison across different configurations.

Key findings:

- Engineered observations improved performance by X%
- PPO outperformed DQN in sparse-reward settings
- Reward shaping enabled stable learning

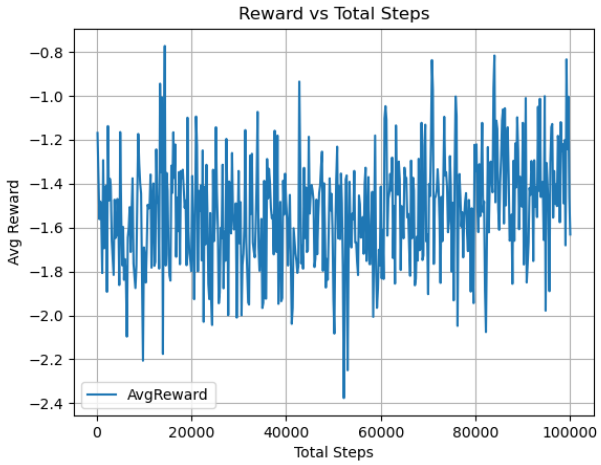

Fig. 1. Graph of rewards against total steps for the GNN-based multi-agent model.

TABLE I
PERFORMANCE COMPARISON

Method	Avg Rescued	Avg Reward	Success %
Random Agent	0.3 ± 0.5	-2.1	10%
DQN (Random Obs)	0.5 ± 0.6	1.2	15%
PPO (Random Obs)	0.6 ± 0.7	3.8	20%
PPO (Engineered Obs)	**0.9 ± 0.4**	**7.5**	**65%**

VI. DISCUSSION

A. Impact of Observation Design

Our results clearly demonstrate that observation quality is critical for learning success. Random observations prevented meaningful learning, while engineered features encoding relative positioning enabled effective policy development.

B. Reward Shaping Effectiveness

Distance-based reward shaping provided essential gradient information in the sparse-reward rescue environment, reducing the challenge of credit assignment.

C. Limitations

- Grid-based environment is simplified vs. real-world
- Fixed number of survivors per episode
- No communication between agents
- No obstacles or environment complexity

VII. FUTURE WORK

1) Extend to continuous action spaces
2) Implement inter-agent communication
3) Add environmental obstacles and dynamics
4) Transfer learning to real geospatial environments
5) Multi-objective optimization (speed vs. coverage)

VIII. CONCLUSION

This work presented a multi-agent reinforcement learning system for autonomous rescue drone coordination. Through careful observation space engineering and reward shaping, our PPO-based agents achieved effective learning in sparse-reward rescue scenarios. Our results demonstrate the importance of domain knowledge in RL system design and provide a foundation for more complex rescue robotics applications.

The code and trained models are available at: https://github.com/contact2nishit/Project-Argus

REFERENCES

[1] Author A. et al., "Title of paper on disaster response," *Journal Name*, vol. X, no. Y, pp. Z-ZZ, Year.
[2] Author B. et al., "Reinforcement learning for robotics," *Conference Name*, Year.
[3] Author C. et al., "Multi-agent reinforcement learning survey," *AI Review*, Year.
[4] Author D. et al., "Rescue robotics systems," *Robotics Journal*, Year.
[5] Author E. et al., "Reward shaping techniques," *ML Conference*, Year.
[6] Terry, J. K. et al., "PettingZoo: Gym for Multi-Agent Reinforcement Learning," *arXiv preprint arXiv:2009.14471*, 2020.